CS 5100 — AI Agent for 2048: Final Progress Report

Rida Sameer & Shreyans Jain

March 2026

1. Project Overview

The goal of this project is to build and compare two AI agents that autonomously play the game 2048:

1. **Expectimax Search Agent** — a classical search-based agent using hand-crafted heuristics (baseline)
2. **TD Learning with N-Tuple Networks** — a reinforcement learning agent based on Jaskowski (2018) (primary contribution)

GitHub: https://github.com/ShreyansJa1n/CS5100-AI-2048

The game is modeled as a **fully observable, nondiscounted discrete MDP** with approximately 4.7×10^{21} states, making tabular methods infeasible. Both agents use either search or function approximation to handle this enormous state space.

2. Progress Summary

Since the initial progress report, we have completed the following milestones:

Milestone	Status
Game engine + GUI (`game2048.py`)	Done
Expectimax search agent (`agents/expectimax_agent.py`)	Done
Evaluation harness (`evaluate.py`)	Done
TD learning agent (`agents/td_ntuple_agent.py`)	Not started
Training script (`train_td.py`)	Not started
Hint mode (UI overlay)	Not started
Final experiments & comparison	Not started

We estimate the project is approximately **50% complete**. The core infrastructure (game engine, evaluation pipeline) and the first agent (expectimax) are fully functional, providing a solid foundation for the remaining work.

3. Expectimax Search Agent

File: `agents/expectimax_agent.py`

3.1 What It Does

The expectimax agent is a tree search agent that looks several moves ahead before deciding on the best direction to play. It requires no training or learned weights — it is purely algorithmic and serves as the baseline against which the TD learning agent will be compared.

3.2 How the Tree Search Works

The game tree alternates between two kinds of nodes:

- **MAX nodes** — the agent picks the best direction (up, down, left, right)
- **CHANCE nodes** — a tile spawns at a random empty cell ($P(2) = 0.9$, $P(4) = 0.1$)

At a search depth of 3, the agent simulates three full agent moves, each followed by a stochastic tile spawn, and evaluates the resulting board with a heuristic function. The direction with the highest expected value at the root is chosen.

3.3 Heuristic Function

At leaf nodes, the board is scored by a weighted combination of four signals:

1. **Empty cell bonus** — $\log(\text{empty_count} + 1) \times 2.7$ rewards boards with more free cells, providing greater flexibility for future moves.
2. **Monotonicity** — rewards boards where tile values decrease monotonically along rows and columns, encouraging the well-known "snake" or "corner" strategy.
3. **Smoothness** — penalizes large differences between adjacent tiles (using $\log_2$ values), since smooth boards are easier to merge.
4. **Corner bonus** — awards $\log_2(\text{max_tile})$ if the highest-value tile occupies any corner, a proven strategy in 2048.

3.4 Performance Optimization

The main performance lever is `_MAX_CHANCE_CELLS`, which caps the number of empty cells considered at each chance node. With the default value of 2:

Depth	Leaf evaluations per move	Approximate time per game
1	4	< 1s
2	64	~5s

Depth	Leaf evaluations per move	Approximate time per game
3	1,024	~60–90s
4	16,384	~15min+

The default configuration is depth 3 with 2 chance cells, striking a balance between quality and speed.

3.5 Pure Board Simulation

The agent does not use the `Game2048` engine internally. Instead, it implements its own pure-function board simulation (`_apply_move`, `_merge_line`) that operates without side effects. This is critical because the search needs to simulate thousands of hypothetical moves without mutating state, and pure functions are faster and easier to reason about.

4. Evaluation Harness

File: `evaluate.py`

4.1 What It Does

The evaluation harness runs any agent for N games headlessly (no GUI) and reports comprehensive statistics. It is designed to be agent-agnostic — any function with the signature `fn(state_dict) -> direction` can be evaluated.

4.2 CLI Interface

The harness supports the following command-line options:

Flag	Description
`--agent`	Agent to evaluate: `expectimax` or `random`
`--games`	Number of games to run (default: 100)
`--depth`	Expectimax search depth (default: 3)
`--chance-cells`	Chance node branching cap (default: 2)
`--verbose`	Print board + chosen move at every step
`--gui`	Watch the agent play in the Tkinter window
`--gui-delay`	Milliseconds between moves in GUI mode

4.3 Example Usage

```
# Run the expectimax agent for 100 games with default settings (depth=3)
python evaluate.py
```

```python
# Run 50 games with a deeper search (depth 4) and more chance cells
python evaluate.py --agent expectimax --games 50 --depth 4 --chance-cells 4

# Compare against the random baseline
python evaluate.py --agent random --games 200

# Debug a single game step-by-step (prints board + move at every step)
python evaluate.py --verbose --games 1

# Watch the agent play live in the GUI
python evaluate.py --gui

# GUI mode with a slower animation speed
python evaluate.py --gui --gui-delay 200
```

4.4 Metrics Reported

After all games complete, the harness prints:

- **Average, median, and best score** across all games
- **Standard deviation** of scores
- **Average moves per game** — a proxy for how long the agent survives
- **Win rate** — percentage of games reaching the 2048 tile
- **Max tile distribution** — a histogram showing what percentage of games reached each tile threshold (128, 256, 512, 1024, 2048)
- **Total time and per-game time** — useful for comparing computational cost

A progress bar (via `tqdm`) shows real-time statistics during batch runs.

4.5 Additional Modes

- **Verbose mode** (`--verbose --games 1`): prints the full board state and chosen direction at every move, useful for debugging agent behavior.
- **GUI mode** (`--gui`): launches the Tkinter window and lets the user watch the agent play live.
- **Random agent baseline**: a built-in random agent that picks uniformly from legal moves, serving as the performance floor.

4.6 Extensibility

The harness uses an agent registry pattern. Adding a new agent (such as the upcoming TD agent) requires only registering a new builder function in `_build_agent()` and adding the name to the `AGENTS` list. The evaluation logic itself requires no changes.

5. Remaining Work

The second half of the project focuses on the TD learning agent and final comparison:

1. **TD Learning with N-Tuple Networks** (`agents/td_ntuple_agent.py`) — implement the n-tuple network as a value function approximator and the afterstate TD(0) learning algorithm. N-tuple networks use fixed patterns of board cells as lookup table indices, providing an efficient alternative to neural networks for this domain.

2. **Training Script** (`train_td.py`) — a standalone script that plays thousands of games headlessly, updating the n-tuple weights after each afterstate transition using the update rule $V(s) \mathrel{+}= \alpha \cdot (r + V(s') - V(s))$. Trained weights will be saved in `.npz` format for portability.

3. **Comparative Evaluation** — run both agents for 100+ games each and compare average score, win rate, max tile distribution, and computational cost. We also plan to test expectimax at multiple depths to measure search depth sensitivity.

4. **Hint Mode** (stretch goal) — a UI feature that overlays the recommended move from whichever agent is available, without auto-playing.

6. Key Reference

Jaskowski, W. (2018). "Mastering 2048 with delayed temporal coherence learning, multistage weight promotion, redundant encoding, and carousel shaping." *IEEE Transactions on Games*, 10(1), 3–14.